

# Reproduction Report: Are Transformers Effective for Time Series Forecasting?

Tu Tran-Anh Nguyen  
People’s Security Academy of Viet Nam

March 28, 2026

## Abstract

This report provides an independent reproduction and robustness analysis of the LTSF-Linear baseline proposed to challenge Transformer-based models in long-term time series forecasting. By conducting experiments under constrained computational environments (CPU-only) and evaluating hyper-parameter sensitivity, I empirically verify the original authors’ claims regarding model efficiency and the inherent temporal information loss in Transformers.

## 1 Introduction

This report presents a comprehensive reproduction of the paper "Are Transformers Effective for Time Series Forecasting?" authored by Ailing Zeng, Muxi Chen, Lei Zhang, and Qiang Xu. The original work was presented at the Thirty-Seventh AAAI Conference on Artificial Intelligence (AAAI 2023). The official implementation provided by the authors is available on [GitHub].

The original authors question the validity of utilizing complex, permutation-invariant Transformer architectures for Long-Term Time Series Forecasting (LTSF) tasks. To demonstrate this, they introduced a set of embarrassingly simple linear models (LTSF-Linear) that surprisingly outperform sophisticated Transformers. The primary objective of this project is to reproduce these core findings, document the implementation challenges (including hardware configurations and missing dependencies), and further explore the method’s robustness by altering the look-back window hyper-parameters and testing on cyclic datasets.

## 2 Reproduction Setting and Implementation

To practically evaluate the resource limits of the proposed models, I intentionally established a strictly **CPU-only** reproduction environment on Ubuntu 22.04 via Zsh. The software stack strictly adhered to the authors’ specifications: **Miniconda**, **Python 3.6.9**, and **PyTorch 1.9.0**.

Experiments were conducted on two highly representative datasets: **Exchange-Rate** (8 variates, highly noisy, aperiodic) and **Electricity** (321 variates, strong daily/weekly periodicity). I evaluated the LTSF-Linear family (DLinear, NLinear) against four prominent Transformer variants (Autoformer, Informer, FEDformer, Pyraformer). Following the original paper’s findings on overfitting, the look-back window ( $L$ ) was set to 336 for Linear models and 96 for Transformers. All models were optimized using the MSE loss function with an Early Stopping patience of 3 epochs.

## 3 Experiments and Results

**Accuracy and Efficiency Comparison** Empirical results (Table 1) confirm that DLinear significantly outperforms Transformer architectures in both predictive accuracy and computational efficiency.

On the Exchange-Rate dataset ( $T = 96$ ), DLinear reduced the MSE by over 45% compared to Autoformer. Furthermore, complex models like Informer and Pyraformer experienced severe accuracy collapse ( $MSE > 0.9$ ).

Table 1: Detailed MSE & Time Comparison across forecasting horizons ( $T$ ). For Exchange-Rate, all models use look-back window  $L = 96$ . For Electricity, Linear models use  $L = 336$  and Transformers use  $L = 96$  to prevent overfitting.

| 2*Model        | Exchange-Rate |               |               |               | Electricity   |               |                |                | 2*Avg. Time/Epoch   |
|----------------|---------------|---------------|---------------|---------------|---------------|---------------|----------------|----------------|---------------------|
|                | $T = 96$      | $T = 192$     | $T = 336$     | $T = 720$     | $T = 96$      | $T = 192$     | $T = 336$      | $T = 720$      |                     |
| <b>DLinear</b> | <b>0.0782</b> | <b>0.1559</b> | <b>0.3069</b> | <b>0.7816</b> | <b>0.1400</b> | <b>0.1537</b> | <b>0.1694*</b> | <b>0.2049*</b> | <b>~2 - 80 secs</b> |
| NLinear        | 0.0856        | 0.1758        | 0.3287        | 0.8859        | -             | -             | -              | -              | ~1 - 3 secs         |
| FEDformer      | 0.1335        | 0.2680        | 0.4524        | 1.1536        | -             | -             | -              | -              | ~650 - 2070 secs    |
| Autoformer     | 0.1427        | 0.2728        | 0.4622        | 1.0885        | 0.2031        | 0.2339        | OOM            | OOM            | ~258 - 1800 secs    |
| Transformer    | 0.5570        | 1.0893        | 1.7954        | 3.0810        | -             | -             | -              | -              | ~166 - 774 secs     |
| Informer       | 0.9594        | 1.2417        | 1.7950        | 2.9320        | -             | -             | -              | -              | ~159 - 530 secs     |
| Pyraformer     | 1.8224        | 1.8735        | 1.9565        | 2.1125        | -             | -             | -              | -              | ~85 secs            |

\*Note: Electricity metrics at  $T \in \{336, 720\}$  for DLinear are derived from the best Test Loss prior to OOM interruptions during testing.

Autoformer at  $T \in \{336, 720\}$  crashed during training (OOM). Models marked with (-) were not evaluated on this dataset to conserve CPU compute limits.

**Further Exploration: The Look-back Window Paradox** Contrary to the standard deep learning assumption that "more data is better," my tests verify a critical paradox. As shown in Table 1, DLinear achieves an MSE of 0.7816 at  $T = 720$  using  $L = 96$ . However, increasing the `seq_len` to 720 actually degraded the MSE to 1.1266. This empirically proves that for financial data with a low signal-to-noise ratio, extending the historical window merely introduces unhelpful noise, forcing the model to overfit localized random walks rather than capturing true trends.

## 4 Own Observations and Criticisms

My CPU-constrained reproduction exposed severe practical limitations in Transformer-based LTSF solutions:

- **Implementation Flaws:** The official repository contains hardcoded `.cuda()` allocations causing immediate CPU crashes, requiring manual refactoring to `.to(corr.device)`. Furthermore, the environment setup lacked declarations for critical implicit dependencies (e.g., `einops`, `sympy`), requiring manual intervention to successfully run the Transformer architectures.
- **The Complexity Paradox:** Complex architectures yielded an indefensible cost-to-performance ratio. DLinear required only ~2 seconds per epoch. Conversely, FEDformer and Autoformer consumed 10 to 35 minutes per epoch while delivering strictly inferior accuracy. Furthermore, Pyraformer’s multi-resolution pyramidal attention completely distorted the financial time series, resulting in an unusable MSE of 1.8224 at  $T = 96$ .
- **Severe Memory Bottlenecks:** I encountered critical Out-of-Memory (OOM) failures in both execution phases. During *inference*, DLinear was terminated by the Linux OOM Killer at  $T \in \{336, 720\}$  due to massive prediction array aggregations. More critically, during *training*, Autoformer crashed mid-process (Epoch 3) at  $T = 336$  on the Electricity dataset. The memory complexity of self-attention intermediate matrices exhausted system RAM, proving these models are practically unscalable for high-variate, long-horizon tasks on standard hardware.

## References

[1] Zeng, A., Chen, M., Zhang, L., & Xu, Q. (2023) Are transformers effective for time series forecasting?. In *Proceedings of the AAAI conference on artificial intelligence* (Vol. 37, No. 9, pp. 11121-11128).

## A. Technical Appendices and Supplementary Material

### A.1 Hardware and Environment

Experiments were performed on an Ubuntu 22.04 system with 16GB RAM and an 8-core CPU. No GPU was utilized, consistent with my objective to evaluate the practical feasibility of these models in resource-constrained settings.

### A.2 Log Files and Artifacts

Full execution logs, including training loss curves, per-epoch timing, and system resource utilization, are provided to ensure the transparency and reproducibility of my results. These artifacts document the stable performance of DLinear and the specific failure points (OOM) of Transformer-based models on the Electricity dataset.

The complete collection of log files and experimental artifacts is available at the following link:  
**[Full Experimental Logs on Google Drive]**

### A.3 Repository Adjustments

To ensure the original codebase could execute correctly in a CPU-only environment, the following essential modifications and setups were implemented:

- **Device Agnostic Refactoring:** The official implementation contained several hardcoded `.cuda()` allocations (notably in `models/Autoformer.py` and `exp/exp_main.py`), which triggered runtime crashes on systems without a GPU. I refactored these instances to use `torch.device` detection, ensuring tensors are correctly mapped to the CPU.
- **Dependency Management:** I identified and manually installed several implicit Python dependencies required by the Transformer architectures (such as `einops` and `sympy`) that were missing from the initial environment setup.
- **Configuration Tuning:** Execution scripts were modified to strictly utilize the `-use_gpu False` flag.